

UDP/TCP Echo Application

Computer Networks project, A.A. 2025/26

Feri Alessandro, 7081870

1. Introduction

For this assignment I built a small command-line application in Rust that can work both as a client and as a server. The goal was to cover the required networking features with something easy to run and test from the terminal.

The program supports both UDP and TCP. In server mode it listens on a port chosen at run time, receives data, and sends an echo reply. In client mode it connects or sends to a host and port chosen at run time, transmits a message, and waits for the reply. The number of messages to send is also configurable at run time.

Besides the basic echo behavior, I added a few extra options that were useful during testing: payload transformations on the server side, an optional RTT measurement based on a timestamp included in the payload, and a TCP option to disable Nagle's algorithm.

2. Program Structure

The executable has two subcommands:

```
server <protocol> <port>
client <protocol> <host> <port> <message>
```

The protocol argument can be udp or tcp. On the client side, --count selects how many messages are sent. On the server side, --transform chooses how the reply is modified before it is sent back. The available transforms are none, upper, and reverse.

The UDP implementation is straightforward. The server binds a UdpSocket, receives one datagram, optionally transforms its payload, and sends one datagram back to the sender. Since UDP preserves message boundaries, one datagram naturally corresponds to one application message.

TCP needs a bit more care because it is stream-based. A read() call does not necessarily return one whole message, so the application uses a simple framing rule:

During implementation I initially treated TCP reads as if each read would contain one complete message. That approach was not reliable, because TCP is stream-based and does not preserve message boundaries. After noticing this issue, I changed the design and added an explicit length field before each payload.

```
[4-byte big-endian payload length][payload bytes]
```

On receive, the program first reads the 4-byte length field and then reads exactly that many payload bytes. This is enough to reconstruct each application message correctly and makes the TCP side interoperable with any other implementation that follows the same rule.

3. RTT Measurement

The optional --rtt flag enables round-trip time measurement. When this mode is active, the client prepends an 8-byte timestamp to the payload before sending it. The server sends those first 8 bytes back unchanged and applies any transformation only to the user message that follows.

The timestamp format used in the code is NTP64: 32 bits for the seconds field and 32 bits for the fractional part. In the implementation, the timestamp is produced starting from the system Unix time and converting it with the standard 1900-to-1970 offset.

For each valid reply, the client prints the RTT and the received message. At the end of the run it prints a short summary:

```
Avg RTT
Max RTT
Sent
Received
Unanswered
```

Unanswered is simply Sent - Received. For UDP this can mean packet loss or timeout. For TCP it usually means that the expected application-level reply was not received, even though the transport itself handles retransmissions internally.

4. Test Setup

I tested the program both locally and with a remote server on Amazon EC2. For the measurements reported here, the server was running on an EC2 instance in the Frankfurt region and the client was running from my machine. I opened UDP port 8888 and TCP port 9999 in the instance security group and used the public IP 52.29.154.138.

The main RTT tests were run with 1000 messages per execution and `--rtt` enabled. For the basic UDP and TCP measurements I started two client runs in parallel, so the server had to handle two clients at the same time.

Example commands:

```
cargo run -- server udp 8888 --rtt
cargo run -- client udp 52.29.154.138 8888 ciao --count 1000 --rtt

cargo run -- server tcp 9999 --rtt
cargo run -- client tcp 52.29.154.138 9999 ciao --count 1000 --rtt
```

I also repeated the TCP tests with `--no-delay`, which enables `TCP_NODELAY`. I wanted to check whether disabling Nagle's algorithm had any visible effect on small echo messages.

5. Results

The first comparison is between standard UDP and TCP runs.

Protocol	Run	Sent	Received	Unanswered	Avg RTT	Max RTT
UDP	1	1000	1000	0	20.012 ms	25.005 ms
UDP	2	1000	1000	0	19.327 ms	23.766 ms
TCP	1	1000	1000	0	20.227 ms	77.253 ms
TCP	2	1000	1000	0	19.234 ms	75.241 ms

In these runs the average RTT is very similar for the two protocols, roughly around 19-20 ms. There is no major difference in the mean value. The clearest difference is in the maximum RTT: TCP shows a few larger spikes, while UDP stays more regular in these measurements.

The TCP results with and without `TCP_NODELAY` are shown below.

TCP mode	Run	Sent	Received	Unanswered	Avg RTT	Max RTT
Default	1	1000	1000	0	20.227 ms	77.253 ms
Default	2	1000	1000	0	19.234 ms	75.241 ms
No delay	1	1000	1000	0	20.091 ms	24.458 ms
No delay	2	1000	1000	0	19.233 ms	26.423 ms

Here the average RTT still stays almost unchanged, but the worst-case RTT becomes much lower when `TCP_NODELAY` is enabled. I would not generalize this result too much, because it depends on the operating system, packet timing, and the specific network path, but in this test it was clearly visible.

6. Echo Transform Checks

To verify the application logic beyond the plain echo case, I also tested the two payload transformations implemented by the server. The message sent by the client was `ciao`.

Protocol	Transform	Received payload	Sent	Received	Unanswered
UDP	upper	CIAO	1000	1000	0
UDP	reverse	oaic	1000	1000	0
TCP	upper	CIAO	1000	1000	0
TCP	reverse	oaic	1000	1000	0

These checks confirm that the transformation code works correctly for both protocols. They also confirm that the RTT timestamp handling is correct, because only the message content is modified while the initial timestamp bytes are echoed back unchanged.

7. Interoperability

The assignment encourages interoperability tests with other groups. I did not manage to complete that part because I did not have another group available at the same time, but the application was written in a way that should make external testing simple.

For UDP the behavior is immediate: one datagram in, one datagram out. For TCP, the other side has to respect the 4-byte big-endian length prefix described above. Without a shared framing rule, the two peers may establish a connection successfully but still disagree on where each application message begins and ends.

8. Conclusion

The final program satisfies the required points of the assignment: configurable ports, UDP send and receive, TCP client/server behavior, configurable message count, and echo replies for both protocols. The extra RTT feature also works and was useful for comparing the two transport choices.

From the tests collected here, UDP and TCP behave similarly in terms of average RTT on this path, while TCP shows a few larger delays in the default configuration. Enabling `TCP_NODELAY` did not change the average much, but it reduced the largest observed RTT values in these runs.

Overall, this was a small project, but it was enough to show an important practical difference: UDP preserves message boundaries on its own, while TCP requires the application to define them explicitly.